The Basics with Scikit-Learn

R. Kumar*
*Department of Mechanical Engineering, University of Tennessee at Chattanooga Chattanooga, TN USA

I. Introduction

Language identification is a fundamental problem in natural language processing (NLP) and text classification [1]. It involves automatically determining the language of a given text sample using computational methods. Even short pieces of text, such as individual words, often contain linguistic patterns that can help distinguish one language from another. Machine learning techniques provide effective tools for detecting these patterns and building predictive models.

In this assignment, a supervised machine learning approach is used to classify words based on their language. The dataset consists of word lists from two provided text files containing English and German words. In addition, a third language is incorporated to extend the classification task to a multi-class problem. Each language is assigned a numerical label for model training, where English is represented by 0, French by 1, German by 2 and the additional language by another unique label.

To maintain consistency and simplify feature extraction, the dataset is filtered to include only five-letter words. These words are then preprocessed and converted into numerical representations suitable for machine learning algorithms. The resulting dataset is divided into training and testing subsets to evaluate model performance.

Three different machine learning models are implemented and compared in this study: the K-Nearest Neighbors (KNN) classifier, the Support Vector Machine (SVM), and the Multi-Layer Perceptron (MLP) neural network. Each model uses a different strategy to learn patterns in the data and perform classification. By training and evaluating these models on the processed dataset, the assignment explores how different algorithms perform in identifying languages based on short word patterns. The goal of this work is to build accurate language classification models, evaluate their performance on unseen data, and compare the effectiveness of different machine learning approaches for this task[2].

II. Methodology

This study applies supervised machine learning techniques to classify words based on their language. The dataset consists of English and German word lists provided in text files, along with an additional third language added to extend the classification task to a multi-class problem. Each language is assigned a numerical label (English = 0, French = 1, and the third language German = 2). To maintain consistency in feature representation, the dataset is filtered to include only five-letter words. The data is then preprocessed and converted into numerical features suitable for machine learning algorithms. Finally, the dataset is divided into training and testing subsets to evaluate model performance.

Three machine learning algorithms are used in this work: K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Multi-Layer Perceptron (MLP). Each model learns patterns in the processed dataset to classify words according to their language.

A. Data Preprocessing and Feature Representation

Text data must be transformed into numerical form before it can be used by machine learning algorithms. In this work, three feature extraction approaches are employed: ordinal encoding, normalized ordinal encoding, and character n-grams.

1) Ordinal Encoding (ord): Each character in a word is converted to its ASCII numerical value using the `ord()` function. If a word consists of characters $c_1, c_2, \ldots, c_n$, the feature vector is defined as

$$x = [\text{ord}(c_1), \text{ord}(c_2), \ldots, \text{ord}(c_n)] \tag{1}$$

For a five-letter word, the resulting feature vector has five numerical elements corresponding to the ASCII values of each character.

2) Normalized Ordinal Encoding: To ensure that the numerical features remain within a consistent range, normalization is applied using

$$x_i^{norm} = \frac{x_i - x_{min}}{x_{max} - x_{min}} \tag{2}$$

where x_i is the ASCII value of the character and x_{min} and x_{max} represent the minimum and maximum values in the dataset.

3) Character n-grams: Character n-grams capture sequential patterns within words by extracting contiguous character sequences of length n. For a word of length L, the number of generated n-grams is

$$N = L - n + 1 \tag{3}$$

For example, a 2-gram representation of the word *house* produces the sequence:

$$\{ho, ou, us, se\}$$

These patterns help capture language-specific spelling structures that improve classification performance [3], [4].

The dataset is split into training and testing sets using random seeding, with 20% of the data allocated for testing. The training set is used to train the machine learning models, while the testing set is used to evaluate the accuracy of the trained models for this particular case.

B. K-Nearest Neighbors (KNN)

1) Basic Working Principle: The K-Nearest Neighbors algorithm is a non-parametric classification method that assigns a class to a data point based on the majority class among its nearest neighbors in the training dataset [5]. Instead of building an explicit model during training, KNN stores the training data and performs classification during prediction.

Given a query sample x, the algorithm computes its distance from all training samples and selects the k nearest points.

The most common distance metric used is the Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \tag{4}$$

Another commonly used metric is the Manhattan distance:

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i| \tag{5}$$

The predicted class label is determined using majority voting:

$$\hat{y} = \text{mode}(y_1, y_2, \ldots, y_k) \tag{6}$$

where $y_1, y_2, \ldots, y_k$ are the labels of the nearest neighbors.

2) Hyperparameters: The performance of the KNN classifier is highly dependent on several hyperparameters that control how neighbors are selected and how distances are computed.

- **Number of neighbors** (k): This parameter determines the number of nearest training samples used to classify a new data point. If k is too small, the model becomes sensitive to noise and may overfit the training data. Conversely, a very large value of k may oversmooth the decision boundary and reduce the model's ability to capture local patterns. In practice, k is often selected through cross-validation.
- **Distance metric**: The distance metric determines how similarity between feature vectors is measured. Common distance metrics include Euclidean distance

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \tag{7}$$

  and Manhattan distance

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|. \tag{8}$$

  The choice of metric influences how the algorithm interprets similarity between words represented as numerical vectors.
- **Weight function**: The weight parameter determines how neighboring points contribute to the classification decision. When `weights = uniform`, each neighbor contributes equally to the final prediction. When `weights = distance`, closer neighbors are assigned larger weights, giving them greater influence in the classification decision.
- **Search algorithm**: KNN supports multiple algorithms for finding nearest neighbors, including `brute`, `kd_tree`, and `ball_tree`. Tree-based approaches improve computational efficiency when dealing with moderate-sized datasets.
- **Leaf size**: This parameter controls the number of data points stored in each node of the tree structure used by KD-tree or Ball-tree algorithms. Larger leaf sizes may reduce tree depth but can increase search time.

3) Limitations: KNN requires computing distances between the query sample and all training samples, which can be computationally expensive for large datasets. It is also sensitive to feature scaling and irrelevant features.

C. Support Vector Machine (SVM)

1) Basic Working Principle: Support Vector Machines aim to find an optimal hyperplane that separates data points belonging to different classes while maximizing the margin between them [6]. The margin represents the distance between the hyperplane and the closest training samples, known as support vectors.

The decision function of a linear SVM is given by

$$f(x) = w^T x + b \tag{9}$$

where w is the weight vector and b is the bias.

The classification decision is

$$\hat{y} = \text{sign}(w^T x + b) \tag{10}$$

The optimization problem solved by SVM is

$$\min_{w,b} \frac{1}{2} ||w||^2 \tag{11}$$

subject to

$$y_i(w^T x_i + b) \geqslant 1 \tag{12}$$

For nonlinear classification, kernel functions are used. A commonly used kernel is the radial basis function (RBF):

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2) \tag{13}$$

2) Hyperparameters: The effectiveness of an SVM model depends strongly on the selection of hyperparameters that control margin optimization and kernel transformation.

- **Regularization parameter** (C): The parameter C controls the trade-off between maximizing the margin and minimizing classification errors. The optimization problem can be written as

$$\min_{w,b,\xi} \frac{1}{2} ||w||^2 + C \sum_{i=1}^{N} \xi_i \tag{14}$$

  subject to

TABLE I
COMMON DISTANCE METRICS USED IN KNN

Metric	Formula	Usage
Euclidean	$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$	Most common
Manhattan	$d(x,y) = \sum_{i=1}^{n} \lvert x_i - y_i \rvert$	Grid / city-block distance
Minkowski	$d(x,y) = \left(\sum_{i=1}^{n} \lvert x_i - y_i \rvert^p\right)^{1/p}$	Generalized distance (sklearn default)
Cosine	$\cos(\theta) = \frac{x \cdot y}{\lVert x \rVert \lVert y \rVert}$	Text / NLP problems

$$y_i(w^T x_i + b) \geqslant 1 - \xi_i \tag{15}$$

where ξ_i are slack variables that allow some misclassification. A large value of C forces the model to correctly classify more training points, potentially leading to overfitting, while a smaller value encourages a wider margin.

- **Kernel function**: The kernel determines how the input data is mapped into a higher-dimensional feature space. Common kernels include linear, polynomial, and radial basis function (RBF). For nonlinear datasets, the RBF kernel is often used:

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2) \tag{16}$$

- **Kernel coefficient** (γ): The parameter γ determines the influence of individual training samples in the RBF kernel. A small value of γ results in smoother decision boundaries, while large values allow the model to fit more complex patterns.
- **Degree**: When using a polynomial kernel, the degree parameter controls the degree of the polynomial function used to map the data into higher-dimensional space. Larger degrees allow more complex decision boundaries but may increase the risk of overfitting.
- **Tolerance**: This parameter determines the stopping criterion for the optimization process. Smaller tolerance values may improve accuracy but increase computational time.

3) Limitations: SVM models require careful hyperparameter tuning and can be computationally expensive for large datasets.

D. Multi-Layer Perceptron (MLP)

1) Basic Working Principle: The Multi-Layer Perceptron is a feedforward neural network composed of an input layer, hidden layers, and an output layer [7]. Each neuron computes a weighted sum of its inputs followed by a nonlinear activation function.

The output of a hidden neuron is given by

$$h_j = \sigma\left(\sum_{i=1}^{n} w_{ij} x_i + b_j\right) \tag{17}$$

where w_{ij} represents the weight, b_j is the bias, and σ is the activation function.

A common activation function is the Rectified Linear Unit (ReLU):

$$\sigma(z) = \max(0, z) \tag{18}$$

During training, the network minimizes a loss function using backpropagation. For classification problems, the cross-entropy loss is commonly used:

$$L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i) \tag{19}$$

2) Hyperparameters: The performance of an MLP neural network depends on several architectural and optimization-related hyperparameters.

- **Hidden layer size**: This parameter defines the number of neurons in each hidden layer. If the network contains L hidden layers with n_l neurons in layer l, the transformation performed by the network can be expressed as

$$h^{(l)} = \sigma(W^{(l)} h^{(l-1)} + b^{(l)}) \tag{20}$$

where $W^{(l)}$ and $b^{(l)}$ represent the weights and bias for layer l. Increasing the number of neurons allows the network to capture more complex relationships in the data.
- **Activation function**: Activation functions introduce nonlinearity into the neural network. Common activation functions include ReLU, sigmoid, and hyperbolic tangent. ReLU is widely used due to its computational efficiency:

$$\sigma(z) = \max(0, z) \tag{21}$$

- **Learning rate**: The learning rate controls how quickly the model updates its weights during gradient descent. The weight update rule can be written as

$$w^{t+1} = w^t - \eta \nabla L(w) \tag{22}$$

where η is the learning rate and $L(w)$ is the loss function.
- **Maximum iterations**: This parameter determines how many training iterations are performed during optimization. If the maximum number of iterations is too small, the network may not converge to an optimal solution.
- **Solver**: The solver specifies the optimization algorithm used for training the network. Common solvers include stochastic gradient descent (SGD), Adam, and L-BFGS. Adam is commonly used because it combines the advantages of adaptive learning rates and momentum.
- **Regularization parameter** (α): This parameter controls L2 regularization in the network to prevent overfitting by penalizing large weight values.

3) Limitations: MLP models require larger datasets and higher computational resources compared to simpler algorithms. They also require careful tuning of hyperparameters and are often difficult to interpret.

III. Model Configuration and Hyperparameter Settings

In this study, three supervised machine learning models were implemented: K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Multi-Layer Perceptron (MLP). The hyperparameters for each model were selected based on empirical testing to achieve stable performance on the language classification task.

1) K-Nearest Neighbors Configuration: The KNN classifier was implemented using the `KNeighborsClassifier` from the Scikit-learn library. The following hyperparameter configuration was used:

- **Number of neighbors ($k = 20$):** The classifier considers the 20 closest training samples when predicting the class of a new word. A relatively larger value of k helps reduce sensitivity to noise and improves generalization by smoothing the decision boundary.
- **Weight function (`uniform`):** Each of the k neighbors contributes equally to the classification decision. The predicted class label is determined by majority voting among the nearest neighbors.
- **Distance metric (`Manhattan`):** The Manhattan distance metric was used to measure similarity between feature vectors. This metric is defined as

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i| \tag{23}$$

  where x and y represent two feature vectors. Manhattan distance is often effective for text-based numerical representations.
- **Search algorithm (`KD-tree`):** A KD-tree data structure was used to accelerate nearest neighbor searches. Tree-based structures significantly reduce computational cost during prediction compared to brute-force search.
- **Leaf size (30):** This parameter controls the number of samples stored in each leaf node of the KD-tree. It influences both the tree construction time and query performance.

2) Support Vector Machine Configuration: The SVM classifier was implemented using the `SVC` class from the Scikit-learn library with the following hyperparameter settings:

- **Kernel (`RBF`):** The radial basis function (RBF) kernel was used to capture nonlinear relationships between features. The RBF kernel is defined as

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2) \tag{24}$$

  where γ controls the spread of the kernel.
- **Regularization parameter ($C = 10$):** The parameter C controls the trade-off between maximizing the margin and minimizing classification errors. Larger values of C penalize misclassification more strongly and allow the model to fit the training data more closely.
- **Kernel coefficient ($\gamma = 0.001$):** The γ parameter determines the influence of individual training samples in the feature space. Smaller values lead to smoother decision boundaries and improved generalization.

3) Multi-Layer Perceptron Configuration: The neural network model was implemented using the `MLPClassifier` from the Scikit-learn library. The architecture and training parameters were defined as follows:

- **Hidden layer structure ($(256, 128)$):** The neural network contains two hidden layers with 256 and 128 neurons respectively. This layered architecture enables the model to learn complex hierarchical representations of the input features.
- **Activation function (`ReLU`):** The Rectified Linear Unit activation function was used, defined as

$$\sigma(z) = \max(0, z) \tag{25}$$

  ReLU introduces nonlinearity into the network while maintaining computational efficiency.
- **Optimizer (`Adam`):** The Adam optimization algorithm was used for training the network. Adam combines adaptive learning rates with momentum-based optimization, allowing efficient convergence during training [8].
- **Learning rate ($\eta = 0.01$):** The learning rate determines the step size used when updating the network weights during gradient descent. The update rule can be written as

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w) \tag{26}$$

  where η is the learning rate and $L(w)$ represents the loss function.
- **Regularization parameter ($\alpha = 0.01$):** L2 regularization was applied to prevent overfitting by penalizing large weight values. The regularized loss function can be written as

$$L_{reg} = L + \alpha ||w||^2 \tag{27}$$

  where α controls the strength of the regularization.
- **Maximum iterations (1000):** The model was allowed to train for up to 1000 iterations to ensure convergence of the optimization process.
- **Early stopping:** Early stopping was enabled to prevent overfitting by monitoring validation performance during training.
- **Iterations without improvement (25):** Training terminates automatically if the validation score does not improve for 25 consecutive iterations, reducing unnecessary computation and improving generalization.

These hyperparameter settings were selected based on preliminary experimentation and were found to provide stable and consistent performance for the language classification task.

IV. Results and Discussion

The performance of different machine learning models for language classification was evaluated using three feature representations: ORD, ORDN, and N-Gram. The algorithms for each case can be referred from Appendix. A. The classification performance was measured in terms of testing accuracy, and the detailed numerical results are presented in Table II and Fig. 1.

TABLE II
Accuracy comparison of different machine learning models for language classification using different feature representations.

Model	Accuracy		
	ORD	**ORDN**	**N-Gram**
KNN	71.3%	67.27%	65.65%
SVM	72.07%	67.7%	68%
MLP	69.6%	65.5%	72.28%

Among the evaluated models, KNN achieved accuracies of 71.3% using ORD features, 67.27% using ORDN features, and 65.65% using N-Gram features (Table II). The SVM model demonstrated comparable performance, achieving 72.07% accuracy with ORD features, 67.7% with ORDN features, and 68% with N-Gram features. The MLP model performed best with N-Gram features, achieving 72.28% accuracy, while its performance was 69.6% with ORD and 65.5% with ORDN features.

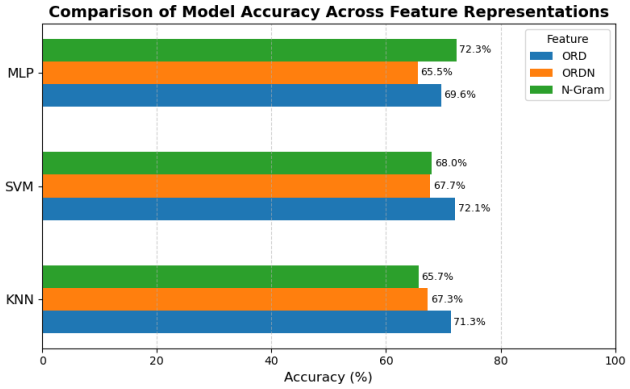

Fig. 1. Comparison of accuracy for different ML models

As illustrated in Fig. 1, the choice of feature representation significantly influences model performance. KNN and SVM models achieved their highest accuracy using ORD features, suggesting that ordinal encoding preserves useful character-level information for these algorithms. In contrast, the MLP model showed superior performance with N-Gram features, indicating that neural networks are more effective at capturing sequential character patterns for language identification.

Overall, the results demonstrate that model selection and feature engineering jointly affect classification performance. Different machine learning algorithms respond differently to feature representations, emphasizing the importance of selecting suitable features to maximize language classification accuracy [2]..

V. Conclusion

This study evaluated the performance of KNN, SVM, and MLP models for language identification using ORD, ORDN, and N-Gram feature representations. The results show that feature selection significantly influences model accuracy. SVM achieved the best overall performance in most cases, while KNN performed well with ORD features and MLP showed superior results with N-Gram features.

These findings demonstrate that machine learning models can effectively identify languages using character-level features. Future work may explore larger datasets and more advanced modeling approaches to further improve classification performance.

Appendix A

The Goggle Collab files for each encoding case are attached below:

- SkiLearn_ord
- SkiLearn_ordn
- SkiLearn_ngram

The Github repository for the project can be found at:

- github

The YouTube video for this assignment could be found using following address:

- YouTube

References

1 Jauhiainen, T., Lui, M., Zampieri, M., Baldwin, T., and Lindén, K., "Automatic language identification in texts: A survey," *Journal of Artificial Intelligence Research*, vol. 65, pp. 675–782, 2019.

2 Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. *et al.*, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

3 Cavnar, W. B. and Trenkle, J. M., "N-gram-based text categorization," in *Proceedings of the Third Annual Symposium on Document Analysis and Information Retrieval (SDAIR)*, Las Vegas, NV, 1994, pp. 161–175.

4 Rahman, M. M. and Ahmed, N. A., "A comparative study on different preprocessing approaches and feature extraction methods for language identification," in *International Conference on Electrical, Computer and Communication Engineering (ECCE)*. IEEE, 2017, pp. 697–702.

5 Cover, T. and Hart, P., "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

6 Cortes, C. and Vapnik, V., "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

7 Rumelhart, D. E., Hinton, G. E., and Williams, R. J., "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

8 Kingma, D. P. and Ba, J., "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, San Diego, CA, 2015.